Seu projeto, explicado

Um guia para entender, operar, reproduzir e manter o T800 Face Follow.

Você criou um sistema para o EngineAI T800 que reconhece pessoas pela câmera, escolhe quem deve acompanhar com base em prioridade e movimenta a cabeça de forma fluida. A visão decide a intenção; o controlador do pescoço executa o movimento físico a 100 Hz.



A ideia principal

O reconhecimento facial não controla o motor diretamente. O app transforma a posição do rosto em velocidade desejada; o safe_neck_controller transforma essa intenção em movimento contínuo, com limites, aceleração, frenagem e watchdog.

Como navegar por este PDF

Quero entender...	Página
O que foi construído e os termos principais	2
Arquitetura geral e arquivos	3
Câmera, detecção e reconhecimento LBPH	4
Prioridade, alvo e fallback	5
Como o rosto vira movimento	6
Controle contínuo do pescoço a 100 Hz	7
Cadastro, dataset e captura guiada	8
Revisão, validação e rebuild	9
Interface, API e operação diária	10
Inicialização, backup e reprodução	11
Mapa das funções e responsabilidades	12
Problemas enfrentados e soluções	13
Diagnóstico rápido de falhas	14
Baseline validada e regras de manutenção	15

Base documentada: versão validada em 23/09/2026. Este guia descreve o comportamento consolidado durante o desenvolvimento e registra também os principais erros, descobertas e decisões arquiteturais.

01 · O que você construiu

Reconhecimento facial, seleção de alvo e acompanhamento físico da cabeça.

O sistema combina visão computacional, uma interface web e controle robótico. Cada pessoa pode ter um perfil com fotos e prioridade. Quando o face follow está ligado, o sistema procura rostos conhecidos e acompanha o alvo mais importante.

Conceito	Significado dentro do projeto
T800	Robô humanoide EngineAI que recebe os comandos físicos da cabeça.
ROS 2	Camada de comunicação com câmera, estados das juntas e comandos do robô.
OpenCV	Processa frames, detecta rostos e executa o reconhecimento LBPH.
LBPH	Modelo de reconhecimento facial; distância menor tende a indicar maior semelhança.
Prioridade	Número menor significa maior prioridade. Prioridade 1 vence 2 e 3.
Face follow	Modo que transforma a posição do rosto em velocidade do pescoço.
Dataset	Pasta de imagens usadas para treinar cada pessoa.
Validação	Aprova uma foto para o dataset; não retreina o modelo imediatamente.
Rebuild / Train	Reconstrói o modelo LBPH usando as imagens atualmente aceitas.

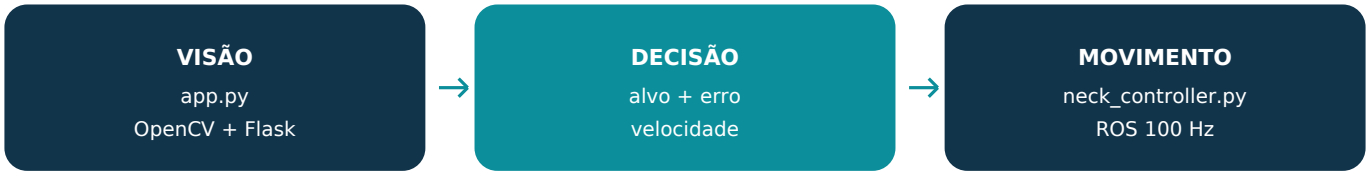
O que o sistema efetivamente faz

- recebe a câmera ROS e disponibiliza vídeo pela interface web;
- detecta um ou mais rostos e tenta identificá-los;
- desenha verde para reconhecidos e vermelho para desconhecidos;
- aplica prioridades e escolhe quem deve ser acompanhado;
- calcula o erro horizontal/vertical em relação à mira;
- envia velocidade desejada ao controlador do pescoço;
- permite cadastrar pessoas, capturar fotos, treinar, revisar e corrigir imagens.

> Uma distinção essencial
>
> Detectar é descobrir que existe um rosto. Reconhecer é decidir de quem é esse rosto. Seguir é usar a posição desse rosto para gerar movimento físico. As três etapas são independentes e podem falhar separadamente.

02 · Arquitetura do código

Cada parte cuida de uma responsabilidade diferente.



Arquivo / componente	Responsabilidade
app.py	Câmera, detecção, reconhecimento, prioridade, face follow, APIs Flask, treinamento e revisão.
templates/index.html	Interface web: vídeo, controles, treino, revisão e correções.
rosto_dados/rostos_conhecidos/	Dataset organizado por pessoa.
rosto_dados/prioridades.json	Mapa nome → prioridade.
rosto_dados/modelo/	Modelo LBPH e labels usados pelo reconhecimento.
rosto_dados/revisao_v8/	Fila de imagens candidatas para revisão humana.
neck_controller.py	Camada segura do pescoço: balance, limites, posição, velocidade, aceleração e publicação ROS.
/motion/joint_override_command	Tópico ROS que recebe comandos das juntas da cabeça.

As duas camadas que não devem ser misturadas

Camada de percepção: responde “onde está o rosto e quem é?”. Pode operar a dezenas de Hz e tolerar pequenas variações de frame.
Camada de controle: responde “como mover o pescoço agora?”. Precisa ser regular, segura e contínua; por isso roda localmente no controlador a 100 Hz.

> Regra arquitetural
>
> A aplicação de reconhecimento não deve publicar diretamente no motor. O fluxo correto é: app → safe_neck_controller → JointOverrideCommand → J23/J24.

03 · Câmera, detecção e reconhecimento

Como um frame JPEG vira um nome na tela.

A câmera principal publica em /manifold/ODIN2/device0/camera0/compressed. O frame chega como JPEG comprimido, é decodificado para BGR e processado pelo OpenCV.

Exemplo conceitual: decodificação

```
raw = np.frombuffer(msg.data, np.uint8)
```

1 · Detecção facial

O Haar Cascade retorna caixas (x, y, largura, altura). Essa etapa responde somente se existe um rosto e onde ele está.

Detector thread-safe

```
boxes = detect_faces_threadsafe(
```

2 · Reconhecimento LBPH

Cada face é recortada, redimensionada para FACE_SIZE e enviada ao LBPH. O retorno é um label numérico e uma distância. Menor distância tende a significar maior semelhança; não é uma porcentagem tradicional de confiança.

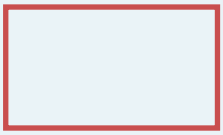
Exemplo simplificado

```
roi = cv2.resize(gray[y:y+h, x:x+w], FACE_SIZE)
```



VERDE

rosto conhecido



VERMELHO

rosto não reconhecido

Thread safety

O Haar apresentou problemas quando usado simultaneamente por vários threads. A solução foi proteger a detecção com lock e concentrar o máximo possível de trabalho em pipelines compartilhados.

04 · Prioridade, alvo e fallback

Quem o robô escolhe quando existem várias pessoas.

Pessoas reconhecidas viram candidatos. Cada candidato guarda nome, prioridade, distância LBPH e caixa do rosto. A regra do projeto é simples: número menor = prioridade maior.

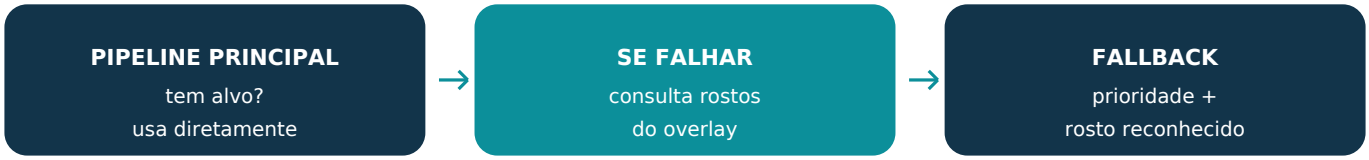
Pessoa	Prioridade	Resultado se todos estiverem visíveis
Guilherme Costa Andrade	1	Preferido
Luiz Sergio	2	Usado se não houver prioridade 1 válida
Benigno Prado	3	Usado se 1 e 2 não estiverem disponíveis

Estrutura simplificada do candidato

```
candidates.append({
    'name': name,
    'priority': prio,
```

Fallback validado

Durante o desenvolvimento houve um caso em que a interface mostrava corretamente o rosto reconhecido, mas /api/status retornava target: null. Isso provou que o overlay e o pipeline principal podiam divergir.



Por que o fallback é útil

O sistema continua seguindo uma pessoa que já foi reconhecida visualmente mesmo que o pipeline principal deixe de publicar o alvo. O pipeline principal continua sendo preferido; o overlay só entra como plano B.

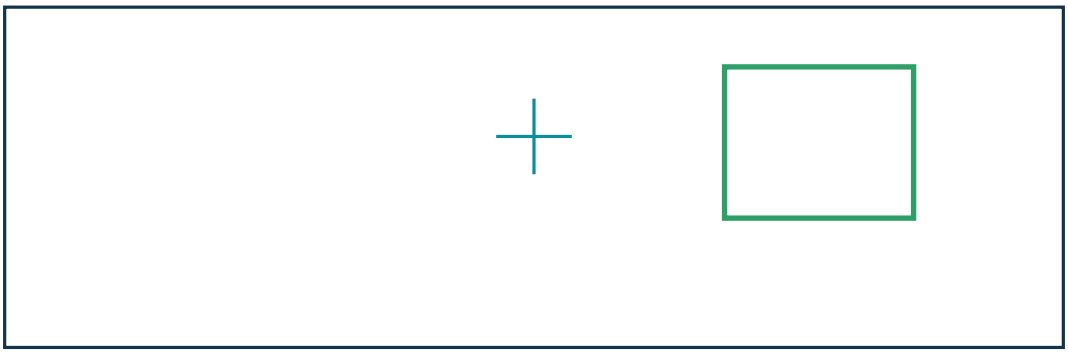
05 · Do rosto até o movimento

Como posição na imagem vira uma velocidade desejada.

Depois de escolher o alvo, calculamos o centro do rosto e o transformamos em erro normalizado. O ponto desejado é aproximadamente X = 50% e Y = 38% da imagem.

Erro normalizado

```
cx = x + fw/2
cy = y + fh/2
ex = (cx - w/2) / (w/2)
```



A mira fica acima do centro geométrico; o rosto pode estar fora e gerar velocidade horizontal/vertical.

Histerese e zona confortável

Não queremos que o robô tente corrigir cada pixel. O sistema usa uma região de parada e uma região um pouco maior para voltar a se mover. Isso reduz o padrão “mexe–para–mexe–para” causado por ruído de detecção.

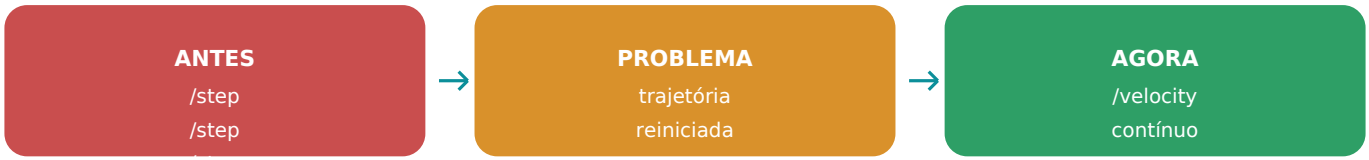
Comportamento desejado

Rosto longe → velocidade maior. Rosto aproximando → velocidade diminui. Rosto dentro da região confortável → velocidade zero. A cabeça fica estável em vez de buscar precisão milimétrica.

06 · Controle contínuo do pescoço

Por que /velocity resolveu a fluidez.

As primeiras versões usavam /step. Cada correção criava uma pequena trajetória independente. O resultado físico era perceptível como uma sequência de passos.



A descoberta que explicou as micro-andadas

Controlador antigo

```
safe_duration = max(0.2, min(5.0, float(duration)))
```

Mesmo quando o app solicitava 15, 25 ou 40 ms, o controlador transformava o movimento em no mínimo 200 ms. Mandar novos steps a cada 25 ms reiniciava uma trajetória que ainda estava em andamento.

Solução: intenção de velocidade

Ideia do modo contínuo

```
POST /velocity
{
  'pitch_velocity': 0.0,
  'yaw_velocity': 0.15
```

O app atualiza a velocidade desejada algumas dezenas de vezes por segundo. O neck_controller integra essa velocidade a 100 Hz, portanto o motor continua se movendo entre dois frames de visão.

Watchdog

Se o app deixar de atualizar a velocidade por algumas centenas de milissegundos, a velocidade desejada volta para zero e o controlador freia. Assim o robô não continua girando se a aplicação travar.

07 · Cadastro e dataset

Como ensinar novas pessoas sem contaminar o modelo.

Cada pessoa possui uma pasta dentro de rosto_dados/rostos_conhecidos/. O cadastro também registra uma prioridade em prioridades.json.

Estrutura simplificada

```
rosto_dados/
├── rostos_conhecidos/
│   ├── Guilherme Costa Andrade/
```

Captura de uma foto

A captura lê o frame, converte para cinza e exige um rosto claro. Em fluxos de cadastro manual, exigir exatamente um rosto reduz o risco de salvar a pessoa errada.

Trecho representativo

```
boxes = detect_faces_threadsafe(gray, 1.1, 5, minSize=(80,80))
if len(boxes) != 1:
```

Captura guiada

- olhar reto para a câmera;
- virar levemente para a esquerda;
- virar levemente para a direita;
- levantar e abaixar o queixo;
- mudar levemente a expressão;
- variar distância próxima, média e mais distante.

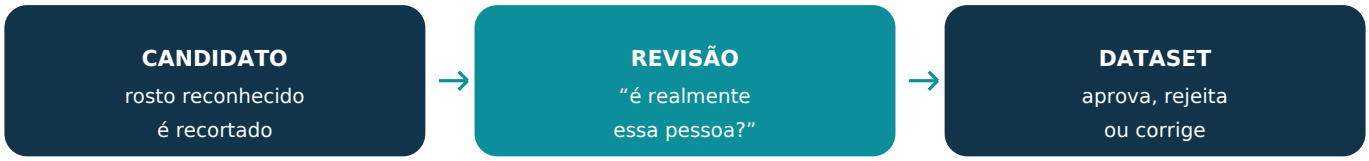
Quantidade não é tudo

Um dataset com muitas imagens quase idênticas pode ser pior que um conjunto menor e variado. O objetivo é ensinar variação real de ângulo, distância, expressão e iluminação.

08 · Revisão, validação e rebuild

Como o sistema aprende novas imagens com supervisão humana.

O sistema pode guardar o recorte exato do rosto que acredita pertencer a alguém. A revisão humana confirma se aquela imagem deve entrar no dataset.



Ação	O que acontece
Validar / SIM	A imagem é salva no perfil correto, normalmente como validated_....jpg.
Não	A candidata é descartada e não entra no treinamento.
Corrigir	A imagem é movida para outra pessoa já cadastrada.
Train / Rebuild	O LBPH é treinado novamente usando o dataset atual.

Regra crítica

Validar uma foto NÃO retreina o modelo imediatamente. Validação altera o dataset. Train/Rebuild é a etapa que gera um novo modelo LBPH e passa a incorporar essas imagens.

Por que essa separação é boa

- evita retreinamento a cada clique;
- reduz concorrência de escrita no arquivo do modelo;
- permite revisar várias imagens antes de um rebuild;
- facilita corrigir erros antes de alterar o reconhecimento em produção.

Problema real já enfrentado

Treinamentos simultâneos chegaram a corromper o YAML do LBPH. A correção foi serializar o treinamento, tratar falhas de carga e mover modelos inválidos para quarentena.

09 · Interface e APIs

O que um operador precisa saber para usar o sistema.

A aplicação Flask atende na porta 3110. O controlador do pescoço atende localmente na porta 5055. A interface web reúne vídeo, status, treinamento, revisão e correções.

Endpoint do app	Uso
GET /	Abre a interface web.
GET /video_safe	Stream MJPEG usado pela interface validada.
GET /api/status	Retorna follow, alvo, prioridade, câmera, posição e pessoas cadastradas.
POST /api/follow	Liga ou desliga o acompanhamento.
POST /api/person	Cria/atualiza pessoa e prioridade.
POST /api/capture	Captura foto pela câmera.
POST /api/upload	Importa imagens.
POST /api/train	Treina o modelo.

Endpoint do pescoço	Uso
GET /health	Confirma que o serviço está vivo.
GET /status	Mostra balance, posição atual, posição real e limites.
POST /balance_mode	Ativa/desativa o modo de controle seguro.
POST /velocity	Atualiza velocidade desejada do tracking.
POST /velocity_stop	Pede parada do modo contínuo.
POST /move /step /center	Mantidos para outros movimentos; não são a base do face-follow validado.

Como interpretar a tela

Quadrado verde = rosto reconhecido. Vermelho = rosto desconhecido. O fato de o quadrado aparecer não garante sozinho que a cabeça se moverá: o follow precisa estar ligado, o alvo precisa chegar ao controlador e o balance mode deve estar ativo.

10 · Inicializar, reproduzir e fazer backup

Fluxo prático para entrar no projeto sem lembrar dezenas de comandos.

1 · Acessar o T800

Entrada rápida

```
ssh ubuntu@192.168.102.133
```

O comando t800 entra na pasta do projeto, carrega ROS 2 Humble, ativa a .venv e exporta as variáveis ROS usadas pela aplicação.

2 · Iniciar a aplicação

Aplicação web e visão

```
python app.py
```

3 · Verificar o controlador

Status do pescoço

```
curl -sS http://127.0.0.1:5055/status | python3 -m json.tool
```

Confirme online: true e balance_mode_active: true. Se o balance estiver desligado, comandos de velocidade podem ser recusados por segurança.

4 · Salvar um checkpoint

Backup recomendado

```
STAMP=$(date +%Y%m%d_%H%M%S)
cp app.py app.py.VERSAO_VALIDADA_$STAMP
cp templates/index.html templates/index.html.VERSAO_VALIDADA_$STAMP
cp /home/ubuntu/scripts-ipq/beniFiles/modules_dashboard/neck/neck_controller.py \
   /home/ubuntu/scripts-ipq/beniFiles/modules_dashboard/neck/neck_controller.py.VERSAO_VALIDADA_$STAMP
```

Baseline congelada

A versão validada deve ser tratada como checkpoint. Qualquer experimento futuro deve começar em uma cópia/branch e só substituir a baseline depois de ser testado.

11 · Mapa das funções

Referência rápida para navegar pelo código.

Local / função	Função prática
CameraStream / leitura da câmera	Receber CompressedImage e expor o frame mais recente.
detect_faces_threadsafe()	Executar Haar Cascade protegido por lock.
vision_loop()	Detectar/reconhecer, escolher candidato e atualizar o erro do alvo.
choose_target()	Aplicar prioridade aos candidatos reconhecidos.
v8_follow_loop()	Transformar erro facial em velocidade desejada; usar fallback se necessário.
v8_start_follow()	Iniciar uma única thread de face follow.
/api/status	Expor estado resumido à interface.
/api/capture / /api/upload	Adicionar imagens ao dataset.
train() / rebuild	Gerar modelo LBPH e labels.
rotinas V8 de revisão	Gerar/mostrar/aprovar/rejeitar/corrigir candidatas.
NeckTrajectoryController.set_velocity()	Receber pitch/yaw velocity com limites.
NeckTrajectoryController.run_control_loop()	Integrar velocidade e publicar J23/J24 a 100 Hz.
NeckHttpHandler	Expor /status, /balance_mode, /velocity e demais rotas HTTP.

Juntas e limites

Junta	Função	Faixa observada
J23	Pitch: cima/baixo	aprox. -30° a +30°
J24	Yaw: esquerda/direita	aprox. -70° a +70°

O comando físico usa JointOverrideCommand com position, velocity, stiffness e damping. Na configuração observada, os ganhos eram KP = 100 e KD = 3.

12 · Problemas enfrentados e soluções

O histórico de erros que levou à arquitetura atual.

Problema observado	Causa / descoberta	Solução validada
Movimento “toc-toc”	/step criava trajetórias independentes e a duração mínima era 200 ms.	Migrar o face-follow para /velocity com integração local a 100 Hz.
40 Hz de steps não melhoraram	Novos comandos reiniciavam trajetórias antigas antes de terminarem.	Separar intenção da visão e movimento físico.
Vídeo muito travado	Múltiplos detectores/revisores rodavam em paralelo e o processo chegou a usar CPU muito alta.	Reduzir pipelines duplicados e compartilhar resultados.
Quadrado reconhecido, mas target null	Overlay e pipeline principal não estavam perfeitamente sincronizados.	Fallback pelo rosto reconhecido do overlay.
Modelo LBPH corrompido	Treinamentos concorrentes escreveram o mesmo YAML.	Lock de treino, rebuild serializado e quarentena.
Modelo corrompido reaparecia	Busca recursiva também encontrava YAML dentro da quarentena.	Restringir busca aos caminhos oficiais do modelo.
Crash do Haar	Cascade usado por vários threads simultaneamente.	detect_faces_threadsafe() com lock.
/velocity devolvia vazio	neck_controller em execução voltou para versão antiga sem endpoint.	Restaurar set_velocity, loop contínuo e endpoint /velocity.
success:false no /velocity	balance_mode_active estava false após reinício.	Ativar balance antes de aceitar movimento.

Lição principal

Não tente corrigir um problema de arquitetura apenas aumentando frequência. O salto de qualidade veio quando o sistema deixou de pensar em “passos” e passou a pensar em “velocidade contínua”.

13 · Diagnóstico rápido

Sintoma → teste → interpretação.

Sintoma	O que conferir primeiro
Quadrado aparece, cabeça não mexe	/api/status: face_follow; /status do neck: balance_mode_active; testar POST /velocity.
/velocity retorna success:false	Balance mode provavelmente desligado.
/velocity retorna vazio / 404	Controlador em execução pode não conter o endpoint novo.
Movimento volta a passos	Procure uso de /step, /move, center() ou nudge_from_face concorrendo com velocity mode.
Reconhece na tela, mas target é null	Verifique fallback do overlay e atualização de v8_follow_error.
Vídeo perde FPS	Meça CPU/threads e procure detectMultiScale duplicado em coletores/revisores.
Reconhecimento piorou após revisão	Lembre que validar altera dataset; confira se houve rebuild e se fotos erradas entraram no perfil.
Erro ao carregar modelo	Verifique YAML, labels e quarentena; não use modelo corrompido.

Comandos rápidos

Diagnóstico básico

```
# status do app
curl -sS http://127.0.0.1:3110/api/status | python3 -m json.tool

# status do pescoço
curl -sS http://127.0.0.1:5055/status | python3 -m json.tool

# teste neutro do endpoint contínuo
curl -sS -X POST -H 'Content-Type: application/json' \
  -d '{"pitch_velocity":0.0,"yaw_velocity":0.0}' \
```

Como separar visão de motor

Se um teste manual de /velocity é fluido, mas o face-follow não reage, o problema está entre detecção → erro → velocidade. Se o teste manual também falha, investigue neck_controller / ROS / hardware antes de mexer no reconhecimento.

14 · Baseline validada

O que deve permanecer estável e como evoluir sem perder o que funciona.

Esta versão foi considerada funcional após resolver o gargalo de vídeo e validar o movimento contínuo da cabeça. Ela deve ser preservada como referência antes de qualquer nova alteração.

Checklist da versão validada

- câmera ROS e stream web funcionais;
- detecção Haar e reconhecimento LBPH;
- verde = reconhecido, vermelho = desconhecido;
- prioridade numérica com menor valor vencendo;
- fallback de tracking quando o alvo principal fica nulo;
- mira vertical mantida em 38%;
- face-follow por velocidade contínua, sem micro-steps;
- safe_neck_controller a 100 Hz com limites e watchdog;
- captura, upload, treinamento e captura guiada;
- revisão humana, correção de perfil e rebuild;
- validação adiciona foto ao dataset, mas não treina automaticamente;
- proteções contra concorrência e modelos corrompidos.

Regra para qualquer próxima versão



O princípio que organiza todo o projeto

Visão entende o ambiente. Reconhecimento diz quem está presente. Prioridade decide quem importa. Tracking transforma a posição em intenção. O safe_neck_controller transforma essa intenção em movimento físico seguro e fluido.

Versão e escopo deste documento

Documento preparado em 23/09/2026 a partir da versão validada durante os testes no T800. Os exemplos de código foram simplificados para ensino, mas refletem as decisões técnicas efetivamente utilizadas no projeto.
Arquivos centrais: app.py · templates/index.html · rosto_dados/ · /home/ubuntu/scripts-ipq/beniFiles/modules_dashboard/neck/neck_controller.py